Field	Details
Project Title	AMR Warehouse Robot Pathfinding Visualizer (Problem 61: Warehouse Robot Navigation)
Team Name / Team No.	05
Year / Semester / Section	II Year B.Tech / I Sem / F Section
Algorithm Paradigm Used	Graph algorithms – shortest-path search on a weighted grid graph: A* (heuristic best-first), Dijkstra's algorithm (greedy, uniform-cost) and Breadth-First Search (baseline)
Programming Language / Tools	Python 3 (NumPy, Matplotlib, Pygame-CE, Streamlit, pandas, Pillow); HTML5 / CSS3 / JavaScript (Canvas) frontend; Python http.server REST API
Faculty Mentor	Alekhya Ma'am & Shivmani Sir

Team Members

S.No	Name of the Student	Roll No.	Role
1	Manthan Mahadev Biradar	25MVCSMR0512	Team Lead
2	Yashvardhan Singh Gaur	25MVCSMR0540	Backend Development
3	Jampala Purnasena	25MVCSMR0491	Backend Development
4	Biswajit Behera	25MVCSMR0472	Frontend Development

Report Overview

This report follows the requested eight-part format. Replace the bracketed cover-page fields and reconcile the implementation/testing statements with the exact build demonstrated during the hackathon. The report intentionally avoids inventing benchmark figures or claiming unverified test outcomes.

Abstract & Introduction

Abstract

CertiVerify is a web-based concept and implementation project designed to make digital certificate verification more reliable, accessible, and efficient. It addresses the difficulty of confirming whether a certificate is genuine when verification depends on manual checks, paper records, or fragmented institutional processes. The proposed platform allows an authorized issuer to register certificate details and associate them with a unique certificate identifier. A verifier can then use the identifier or a QR code to check the certificate's recorded status through a public verification interface. The system is planned around role-based access, certificate lifecycle status, and a persistent database so that issuing and verification workflows can be managed consistently.

Introduction

Educational institutions and training providers issue certificates for courses, events, internships, and qualifications. Employers and other recipients need a dependable way to verify these documents. Paper certificates and static digital files can be copied or altered, while manual verification can take time and require contacting the issuing institution. CertiVerify proposes a centralized digital workflow that helps issuers manage certificate records and enables third parties to verify those records online. The project is developed for the BWT Hackathon using the MERN stack, which supports a React-based interface, an Express/Node.js API layer, and MongoDB data storage.

Problem Statement, Objectives & Scope

Problem Statement

There is no universally consistent, immediate method for a recipient to verify every certificate presented to them. Manual verification can be slow, certificate details may be difficult to trace, and altered or revoked certificates can be mistakenly accepted if status is not checked against an authoritative record.

Objectives

- Build a web application for recording and verifying digital certificates.
- Assign each certificate a unique identifier that can be used for lookup.
- Provide an issuer workflow for creating and managing certificate records.
- Provide a public verification workflow using a certificate ID and, where enabled, a QR code.
- Represent certificate status clearly, including valid, revoked, or expired where applicable.
- Apply authentication and role-based authorization to restricted operations.
- Store certificate metadata in a database and validate incoming API requests.

Scope

The project scope covers certificate metadata management, unique-ID lookup, QR-assisted access to a verification page, role-based access for administrative/issuer operations, and status-based verification. The platform is intended to verify a stored record and its current status; it does not by itself prove the identity of the original certificate holder or replace institutional governance, cryptographic signing, or legal accreditation. Features described as planned or optional should be considered complete only after they have been implemented and tested in the submitted build.

Features & Functional Requirements

Core Features

- User authentication for authorized users.
- Role-based access control for issuer/admin functions and public verification.
- Certificate creation with relevant metadata such as recipient name, certificate title, issuing organization, issue date, and expiry date if applicable.
- Generation or assignment of a unique certificate ID.
- Search and verification by certificate ID.
- QR code that directs a verifier to the certificate's verification page, if QR support is enabled in the build.
- Certificate status handling, including revocation and expiry checks where implemented.
- Issuer/admin dashboard for viewing and managing records.
- Clear success, invalid, revoked, expired, and not-found responses.

Functional Requirements

- FR-1: The system shall allow authorized issuers to create certificate records after validating required fields.
- FR-2: The system shall prevent duplicate certificate identifiers.
- FR-3: The system shall allow a verifier to submit a certificate ID and retrieve the corresponding verification result without exposing restricted administrative data.
- FR-4: The system shall determine the displayed status from the stored record and relevant dates/status fields.
- FR-5: Only authenticated users with the required role shall be able to create, update, or revoke certificate records.
- FR-6: The API shall validate request data and return appropriate errors for invalid input, missing records, and unauthorized actions.
- FR-7: Where QR functionality is enabled, the QR code shall resolve to the same verification workflow as manual ID entry.

Non-Functional Requirements

- Security: enforce server-side authorization, validate inputs, and keep secrets in environment variables rather than source control.
- Reliability: handle missing, malformed, or unavailable records without crashing the application.
- Usability: make the verification result understandable to a non-technical user.
- Maintainability: separate interface, API routes/business logic, and database models.

- Performance: keep lookup operations efficient through indexed unique identifiers where supported.
- Privacy: expose only the minimum certificate information needed for verification.

Algorithms & Methodology

Development Methodology

The project follows an iterative full-stack development approach: identify the verification workflow, define the data model and user roles, build the interface and API, connect persistent storage, and validate the main user journeys. Work is divided into frontend, backend/API, database, security, and testing tasks.

Certificate Registration

1. 1. Receive certificate details from an authorized issuer.
2. 2. Validate required fields and date formats on the server.
3. 3. Generate or validate a unique certificate identifier.
4. 4. Check that the identifier is not already assigned.
5. 5. Save the certificate metadata and initial status to the database.
6. 6. Return a confirmation and, if implemented, a QR code or verification URL.

Verification Algorithm

7. 1. Accept a certificate ID from manual input or a QR-linked URL.
8. 2. Validate and normalize the identifier.
9. 3. Query the database for an exact matching record.
10. 4. If no record exists, return “Not Found” without disclosing private data.
11. 5. If the record is revoked, return “Revoked”.
12. 6. Otherwise, if an expiry date exists and has passed, return “Expired”.
13. 7. Otherwise, return “Valid” and the permitted public certificate details.

Security Methodology

Authorization is enforced on the server, not only by hiding interface controls. Protected endpoints must check authentication and role permissions. Inputs should be validated before database operations, secrets should be stored in environment variables, and errors should avoid revealing credentials or internal implementation details. Verification endpoints should be rate-limited for production use. A production deployment should also use HTTPS, secure session/token handling, logging that excludes secrets, and backup/restore procedures.

Technologies & Implementation

Technology Stack

- **MongoDB:** Document database for certificate records and user/account data.
- **Express.js:** Backend web framework for REST API routes and middleware.
- **React.js:** Component-based frontend for issuer workflows and the public verification interface.
- **Node.js:** JavaScript runtime for backend services.
- **HTML, CSS, JavaScript:** Web interface structure, styling, and client-side interaction.

- **QR code library (if included):** Encodes a verification URL or certificate identifier for quick access.
- **Git/GitHub (if used):** Version control and source-code collaboration.

High-Level Architecture

The React client sends HTTP requests to the Express API. The API validates requests, applies authentication and authorization rules, runs certificate business logic, and reads or writes records in MongoDB. The verification page queries the API using the certificate identifier and displays only the fields permitted for public viewing. This three-tier structure aligns with the common MERN architecture described in MongoDB's MERN documentation.

Implementation Notes

The report describes the intended CertiVerify design and feature set based on the project overview. Before submission, update the stack and feature list to match the exact code that runs in the hackathon build. Mark any unimplemented item as future work rather than claiming it is complete. Keep environment files and database credentials out of the public repository.

Results, Testing & Performance

Expected/Observed Workflow

The target demonstration flow is: an authorized issuer signs in, creates a certificate record, obtains its unique identifier or QR code, and a verifier checks that identifier through the public verification page. The system should return a status based on the database record. Because no measured benchmark results or test logs were provided for this report, no numerical performance claims are made.

Test Plan

- Authentication: confirm valid credentials allow sign-in and invalid credentials are rejected.
- Authorization: confirm unauthenticated users cannot create or revoke records and users without the required role are denied.
- Certificate creation: submit valid data, omit required fields, and attempt duplicate IDs.
- Verification: test valid, unknown, revoked, and expired certificate cases.
- QR flow: scan the QR code and confirm it opens the intended verification page, if implemented.
- API validation: submit malformed identifiers and unexpected field values.
- Data isolation: verify that one user cannot modify records outside their permissions.
- Reliability: test database connection failure and ensure the UI shows a safe error.
- Security configuration: confirm secrets are not committed to Git and sensitive values are not returned in API responses.

Performance Evaluation

For this prototype, performance should be evaluated by recording API response times for certificate lookup, testing with representative records, and checking behavior under repeated requests. A unique index on certificate ID can improve exact-match lookup efficiency. Do not report latency, throughput, test coverage, or pass rates unless they have been measured and recorded.

Results Summary

The design establishes a clear workflow for issuing and checking certificate records and provides a foundation for secure digital verification. Final claims about implemented features, successful test cases, deployment, and measured performance should be aligned with the actual demo and test evidence.

Conclusion, Future Scope & References

Conclusion

CertiVerify addresses a practical trust problem in education and training by providing a structured way to register certificate metadata and verify a certificate record online. The MERN architecture supports a modular implementation in which the frontend, API, and database have distinct responsibilities. The project's reliability depends on accurate issuer records, server-side access controls, clear status handling, and testing of both normal and unauthorized workflows.

Future Scope

- Add institution-level issuer onboarding and verification of issuer identity.
- Add cryptographic digital signatures or tamper-evident hashes for stronger integrity guarantees.
- Provide bulk certificate issuance and CSV/PDF import with validation.
- Add audit trails for issuance, updates, revocation, and verification activity.
- Add multilingual UI, accessibility improvements, and mobile-friendly QR scanning.
- Deploy with HTTPS, managed database backups, monitoring, rate limiting, and automated tests.
- Integrate with institutional student information systems through authenticated APIs.

References

- MongoDB. "MERN Stack Explained." https://www.mongodb.com/resources/languages/mern-stack
- MongoDB Developer. "MERN Stack Example: Employee Records App." https://github.com/mongodb-developer/mern-stack-example
- React documentation. https://react.dev/
- Express documentation. https://expressjs.com/
- Node.js documentation. https://nodejs.org/docs/
- MongoDB documentation. https://www.mongodb.com/docs/